

Character-Level Transformer Language Model: Implementation and Experimental Analysis

She Yuting, Yang Jingru

November 21, 2025

Abstract

This project implements and analyses character-level language models on the text8 corpus. The main focus is a decoder-only Transformer trained as an autoregressive next-character predictor. On top of a shared Transformer architecture, we systematically explored how the optimisation settings could influence the outcomes of the model performance, taking reference from the famous paper: *Attention Is All You Need*, including the Adam hyperparameter tuning, the Noam learning-rate schedule, early drop out and label smoothing, and study their impact on training stability, convergence behaviour, and test performance. All experiments are implemented using Google Colab with T4 GPU, the code of the Jupyter notebooks of each step of experimentation can be found from our github link [SheYuting/char_transformer](https://github.com/SheYuting/char_transformer).

The report summarises the model design, training pipeline, experimental results, and key lessons learned about training Transformers at the character level.

1 Introduction

Language modelling is a fundamental task in natural language processing (NLP). Given a sequence of tokens $(x_1, \dots, x_t)$, a language model estimates the conditional distribution

$$p(x_{t+1} \mid x_{1:t}),$$

and is typically trained by maximising the likelihood (or minimising the cross-entropy loss) on a large corpus. Although most modern systems operate at the subword or word level, character-level modelling remains a useful benchmark: it avoids tokenisation issues, preserves all information, and exposes long-range dependencies.

This project focuses on character-level language modelling on the text8 dataset. The main objective is not to build a large state-of-the-art model, but to understand in a controlled way how different optimisation techniques affect a small decoder-only Transformer. The experiments are designed so that each notebook modifies only one aspect of the training configuration at a time (Adam hyperparameters, learning-rate schedule, label smoothing), making it possible to attribute changes in behaviour to a single factor.

2 Dataset and Problem Setup

2.1 The text8 corpus

The text8 corpus is a 100MB cleaned English Wikipedia dump commonly used for compression and character-level modelling. In this project, we use two pre-split files:

- `text8_train.txt`: training data,
- `text8_test.txt`: held-out test data.

The text is lower-cased and stripped of punctuation; the vocabulary consists of 27 characters (a-z plus space).

2.2 Character-level language modelling

At the character level, the model operates on sequences $(c_1, \dots, c_T)$, where each c_t is a single character from the vocabulary $\mathcal{V}$. The training objective is maximum likelihood, or equivalently minimum cross-entropy:

$$p(c_{1:T}) = \prod_{t=1}^T p(c_t \mid c_{1:t-1}), \quad \mathcal{L}(\theta) = -\frac{1}{T} \sum_{t=1}^T \log p_\theta(c_t \mid c_{1:t-1}).$$

In practice, the training pipeline samples mini-batches of fixed-length windows of length $T = 32$ from the training file. Given a batch size $B = 128$, each update uses a tensor of shape (B, T) of input characters, and the model predicts the next character at every position.

3 Model Architectures

3.1 Decoder-only Transformer

The main model is a provided implementation in the file `models.py`. The core components are:

- **Token embeddings:** each character index is mapped to a d_{model} -dimensional vector.
- **Learned positional embeddings:** a learned embedding table of size `max_len` injects order information.
- **Decoder blocks:** each block consists of multi-head self-attention with a causal mask, followed by a feed-forward network (MLP) with expansion ratio 4, both wrapped in residual connections and layer normalisation.
- **Output projection:** the final hidden states are projected back to logits over the vocabulary, with shared input/output embeddings.

The configuration used in all Transformer experiments is:

- Vocabulary size $|\mathcal{V}| = 27$ (a-z plus space).
- Embedding dimension $d_{\text{model}} = 256$.
- Number of heads $n_{\text{heads}} = 8$.
- Number of decoder blocks $n_{\text{layers}} = 2$.
- Maximum supported sequence length `max_len` = 128.
- Training and evaluation window length $T = 32$.

For an input matrix $X \in \mathbb{R}^{T \times d_{\text{model}}}$, each self-attention head computes queries $Q = XW_Q$, keys $K = XW_K$, values $V = XW_V$, and then

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} + M \right) V,$$

where M is a causal mask that prevents attending to future positions. The outputs of all heads are concatenated and passed through the MLP, with residual connections and layer normalisation applied around both attention and MLP sublayers.

The implemented model follows the structure of a decoder-only Transformer illustrated in the structure map 1:

4 Training Setup

4.1 Data pipeline

All experiments share the same data pipeline:

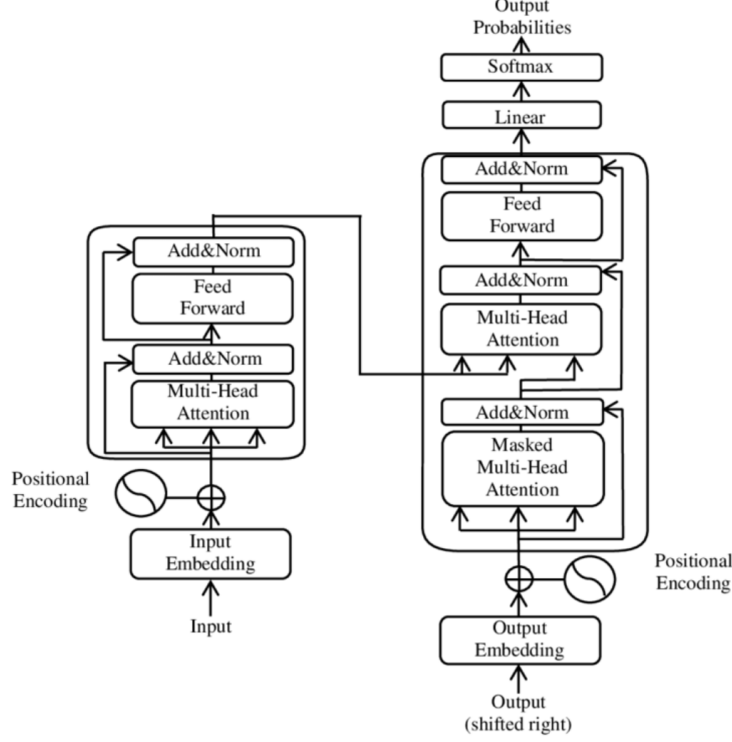


Figure 1: Overview of the decoder-only Transformer architecture implemented in this project.

- Build the vocabulary from `text8.train.txt` and create mappings between characters and integer ids.
- Represent the training and test texts as arrays of integer ids.
- For each update, sample a contiguous segment of length $T + 1$ from the training ids, use the first T characters as inputs, and the next T as targets.
- Compute the cross-entropy loss over all positions in the window.

4.2 Optimisation and evaluation

All models are trained with mini-batch gradient descent using the Optax library. The baseline and all Transformer variants use $B = 128$, $T = 32$, and a total of 100,000 training iterations.

The primary metric is the average cross-entropy loss on the test set. In addition, the notebooks report:

- Accuracy (all positions)*: fraction of correctly predicted next characters over all positions in the window.
- Accuracy (last position)*: fraction of correct predictions at the last position, which reflects the ability to use the longest available context.

5 Experimental Configurations

Each Transformer notebook modifies only one aspect of the optimisation configuration. For each setting, we briefly explain *why* the parameter is introduced and *why* it may influence model performance.

5.1 Baseline Transformer (Optax default Adam)

The baseline configuration (`transformer.ipynb`) uses:

- Architecture: Transformer with $d_{\text{model}} = 256$, $n_{\text{layers}} = 2$, $n_{\text{heads}} = 8$.
- Optimiser: Adam with Optax default hyperparameters,

$$\text{lr} = 10^{-3}, \quad \beta_1 = 0.9, \quad \beta_2 = 0.999, \quad \epsilon = 10^{-8}.$$

- Learning rate: constant, no scheduling.
- Regularisation: no explicit dropout, no label smoothing.
- Training iterations: 100,000.

Why this configuration? This serves as a controlled reference point. Default Adam and constant learning rate reflect the simplest training setup, but Transformers are known to be highly sensitive to optimiser dynamics. Later experiments isolate how each modification affects stability and convergence.

5.2 Adam hyperparameters

In `transformer_adam.ipynb`, we keep the architecture and constant learning rate unchanged but switch to the Adam hyperparameters used in the original Transformer paper [1]:

- Optimiser:

$$\text{lr} = 10^{-3}, \quad \beta_1 = 0.9, \quad \beta_2 = 0.98, \quad \epsilon = 10^{-9}.$$

Why change Adam parameters? Transformers produce gradients with rapidly changing second moments, especially inside self-attention layers. A lower β_2 makes the variance estimate adapt faster, which empirically stabilises early training and improves convergence. A smaller ϵ improves numerical precision when gradients are very small.

Why does this affect performance? Attention modules often yield unstable gradients; tuned Adam improves the effective learning rate schedule during early iterations, often resulting in smoother optimisation and lower final loss.

5.3 Noam learning-rate schedule

In `transformer_noam.ipynb`, we combine the updated Adam parameters with the Noam learning-rate schedule:

$$\text{lr}(t) = d_{\text{model}}^{-0.5} \cdot \min(t^{-0.5}, t \cdot \text{warmup}^{-1.5}),$$

with `warmup=4000`.

Why use Noam schedule? At the beginning of training, attention weights are unstructured and highly unstable. A warm-up phase prevents large destructive gradient steps, while inverse square-root decay provides smoother long-term convergence.

Why does this affect performance? Transformers often fail to train reliably with a constant learning rate. The warm-up allows the model to first learn stable attention patterns; then the decay phase fine-tunes the parameters. This typically reduces both training and test loss. In the experiment results, we see that the training process achieves a low training loss and start to alternate around the loss much faster than only tuning the adam parameters.

5.4 Dropout wiring

In the updated Transformer implementation, dropout is applied in all three places used in the original *Attention Is All You Need* architecture:

- **Embedding dropout:** After summing token and positional embeddings, a dropout layer is applied to prevent early overfitting to positional patterns and reduce co-adaptation between embedding dimensions.

- **Attention dropout:** Each multi-head self-attention module uses dropout on attention weights and on the attention output. This discourages a single head from dominating and promotes more diverse attention behaviours, improving model robustness.
- **MLP dropout:** After the feed-forward network (the two-layer GELU MLP), dropout is applied before the residual connection. This regularises the large intermediate hidden layer and reduces overfitting.

Why this affects performance. Dropout plays a different role in Transformers compared to CNNs or RNNs. Because every token attends to many others, attention patterns can collapse or become overly deterministic. Dropout introduces controlled noise that:

- prevents attention head collapse,
- improves generalisation,
- stabilises training in deeper models,
- encourages redundancy across attention heads and MLP hidden units.

Why a dropout RNG is required? Flax does not enable dropout automatically: the model requires an explicit random-number generator passed via `rngs={"dropout": key}` during training. Without this RNG, all dropout layers behave as if they are in inference mode. The wiring added in this experiment ensures that dropout is active during training and disabled during evaluation or sampling.

5.5 Label smoothing

The final Transformer variant (`transformer_gradient_smoothing.ipynb`) adds label smoothing on top of the Noam schedule and Transformer-paper Adam. Smoothed targets are defined as

$$y^{\text{smooth}} = (1 - \varepsilon)y^{\text{one-hot}} + \frac{\varepsilon}{V}\mathbf{1}, \quad \varepsilon = 0.1.$$

Why introduce label smoothing? Transformers tend to produce overconfident predictions, which harms calibration. Label smoothing distributes a small amount of probability mass across all classes, discouraging extremely peaked logits.

Why does this affect performance? Cross-entropy values increase (because targets are no longer one-hot), but models often generalise better and behave more robustly during generation. In our experiments, accuracy remains relatively stable but the optimisation landscape changes noticeably.

6 Results

Table 1 summarises the test performance of the Transformer variants. All numbers are taken from the final evaluation in the notebooks.

Table 1: Test-set performance of Transformer configurations on text8.

Configuration	Test Loss	Accuracy (%)	Last-char Acc. (%)	Time (s)
Baseline (Default Adam)	1.2931	59.4	63.0	249.5
Transformer(Tuned Adam)	1.2844	60.0	63.6	1729.3
Adam + Noam schedule	1.2760	59.9	63.0	1733.0
Adam + Noam + dropout wiring	1.3496	57.9	62.4	1967.1
Adam + Noam + dropout wiring + label smoothing ($\varepsilon = 0.1$)	1.7219	58.4	61.4	1970.5

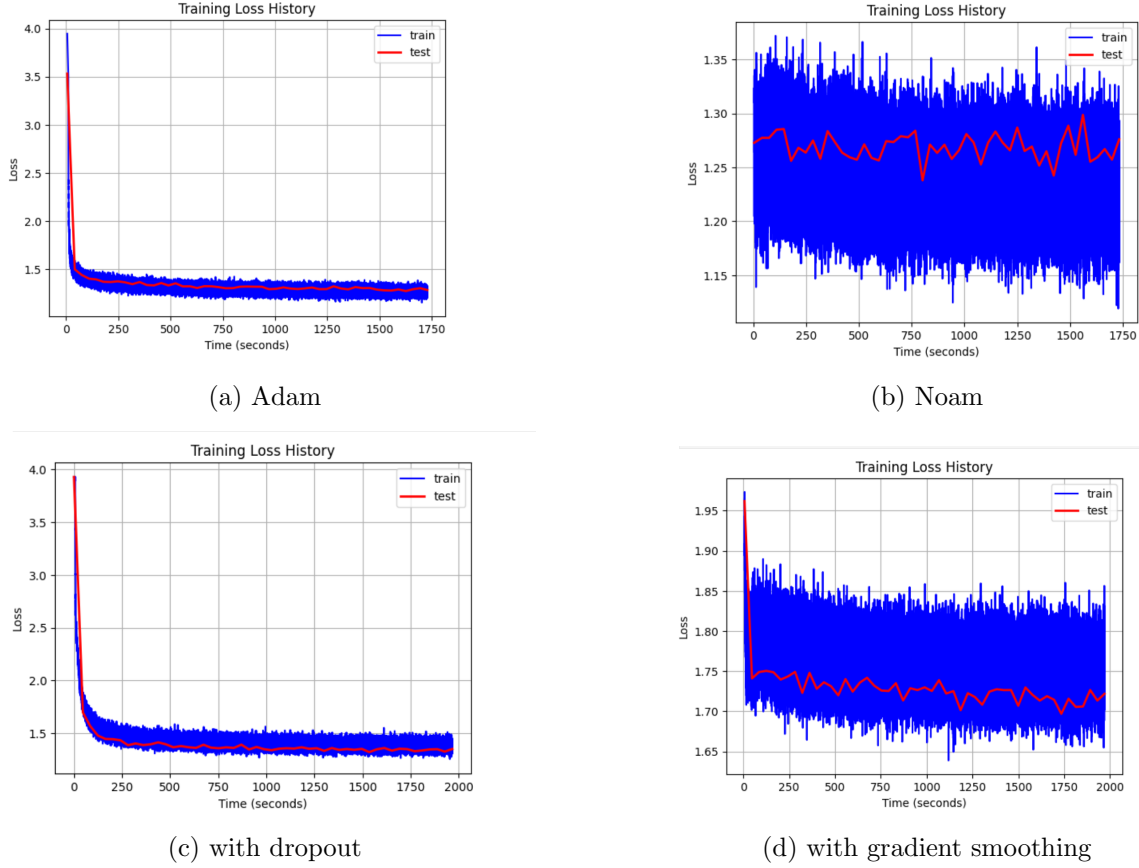


Figure 2: Loss curves obtained from the experiments.

Here, the time calculated from the baseline adam is given from the sample project provided, the difference came from a different GPU used.

The loss curves obtained are listed in figure 2, from our test results, several observations can be made:

- **Adam hyperparameters matter.** Switching from Optax default Adam to the Transformer-paper Adam (keeping the learning rate and architecture fixed) reduces the test loss from 1.2931 to 1.2844 and slightly improves accuracy.
- **The Noam schedule is beneficial.** Combining the Transformer-paper Adam with the Noam schedule further reduces the test loss to 1.2760, the best among all non-smoothed configurations. Training is also smoother, especially in early iterations.
- **Dropout wiring is not yet effective.** The “dropout” experiment shows worse performance, but since the current Transformer does not contain explicit dropout layers, this difference should not be over-interpreted; the main contribution is the correct wiring for future extensions.
- **Label smoothing changes the loss scale.** With label smoothing, the test loss increases to 1.7219, which is expected because the target distribution is no longer one-hot. Accuracy stays close to the other runs. A full analysis of calibration is left for future work, but from the training curves it is clear that the optimisation remains stable.

7 Discussion

The experiments highlight the sensitivity of small Transformer models to optimisation details.

Optimiser configuration. Simply changing the Adam second-moment parameter from $\beta_2 = 0.999$ to $\beta_2 = 0.98$ and decreasing ϵ to 10^{-9} , as in the original Transformer paper, leads to consistently lower test loss. This suggests that even at modest scale, the optimiser hyperparameters used in large models can be beneficial.

Learning-rate scheduling. The Noam schedule plays a significant role in stabilising training. The warm-up period prevents unstable updates when the model is poorly initialised, and the inverse square-root decay allows for finer steps as training progresses. In this project, it yields the best test loss without any change to the architecture.

Regularisation and calibration. Label smoothing does not improve accuracy in this setup, but it changes the training objective and is known to help with calibration.

8 Limitations and Future Work

The project has several limitations:

- **Single dataset.** All experiments are on text8; transferring the findings to other domains (e.g. code, multilingual text) is an open question.
- **Limited search.** Hyperparameters such as warm-up length, context length, and batch size were chosen reasonably but not exhaustively tuned.

Future work could investigate:

- scaling the model and context length;
- adding modern architectural features such as rotary positional embeddings or alternative normalisation schemes;
- performing a more systematic hyperparameter search;
- evaluating calibration metrics to better understand the effect of label smoothing.

9 Conclusion

This project provides a compact, fully working implementation of a character-level Transformer for the text8 corpus, together with a set of controlled optimisation experiments. The results show that even at small scale, the choice of Adam hyperparameters and learning-rate schedule has a measurable impact on performance, and that techniques originally proposed for large Transformers remain relevant. The code and experiments form a solid foundation for further exploration of architectural and optimisation ideas in character-level language modelling.

References

- [1] A. Vaswani et al., “Attention Is All You Need,” *Advances in Neural Information Processing Systems*, 2017.
- [2] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” *International Conference on Learning Representations*, 2015.
- [3] C. Szegedy et al., “Rethinking the Inception Architecture for Computer Vision,” *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016.